Project Report: Real-Time IoT Weather Dashboard

1. Project Objective and System Architecture

1.1 Objective

The primary objective of this project is to develop a **real-time Internet of Things (IoT) telemetry system** capable of reading environmental data (**temperature and humidity**) from a low-cost microcontroller and instantly displaying it on a publicly hosted, zero-latency web dashboard.

The project demonstrates proficiency in:

- Microcontroller programming (ESP8266 / NodeMCU),
- Full-stack serverless architecture, and
- Real-time data streaming using Firebase.

1.2 System Architecture

The system follows a serverless three-tier Publish/Subscribe (Pub/Sub) model:

Layer	Role	Description
Device (Publisher)	NodeMCU (ESP8266)	Reads data from DHT sensor and publishes it to Firebase in real-time.
Cloud (Broker)	Firebase Realtime Database	Stores live sensor data and pushes updates instantly to all connected clients.
Web App (Subscriber)	HTML/JS Dashboard	Subscribes to data changes in Firebase and auto-updates UI in real-time.

2. Materials and Software

2.1 Hardware Components

Component	Model / Description	Purpose
Microcontroller	**NodeMCU (ESP8266)**	Core processing and Wi-Fi communication
Sensor	**DHT22 / DHT11**	Measures temperature (°C) and humidity (%)
Power Source	USB Micro-B Cable	Supplies power and uploads firmware
Connectivity	Jumper Wires / Breadboard	Hardware interfacing

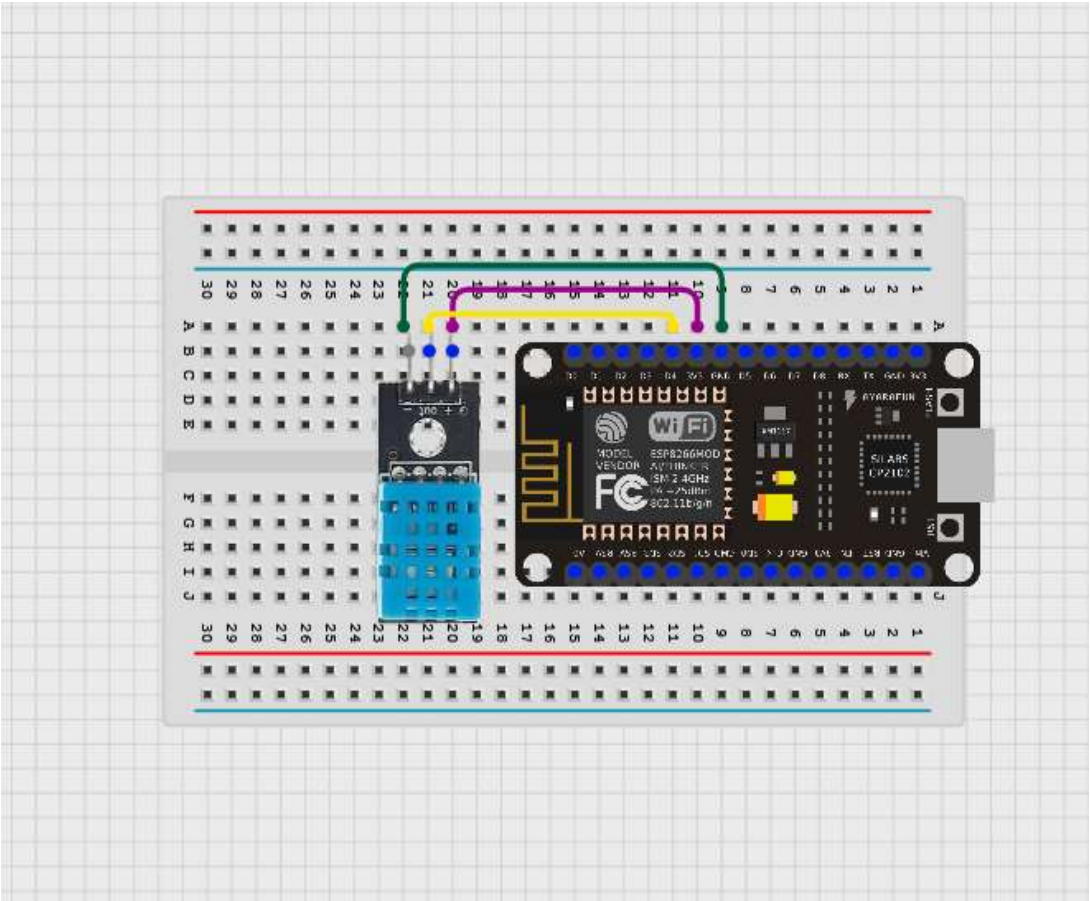
2.2 Software and Libraries

Category	Tool / Library	Purpose
Firmware IDE	Arduino IDE	Programming the NodeMCU in C++
Libraries (C++)	Firebase-ESP-Client, DHT.h	Firebase communication & sensor reading
Backend	Firebase Realtime Database	Cloud storage and data streaming
Frontend	HTML5, JavaScript (ES6 Modules)	Real-time UI updates in the browser
Styling	Tailwind CSS	Dashboard layout and responsiveness

3. Circuit Diagram

3.1 DHT Sensor Connection with NodeMCU

DHT Sensor Pin	NodeMCU Pin	Function
VCC	**3.3V**	Power supply
GND	**GND**	Ground
DATA	**D2 (GPIO 4)**	Digital signal pin



4. Theoretical Concepts

4.1 Real-Time Data Flow (Pub/Sub)

- **Publisher:** NodeMCU sends JSON payloads (temperature, humidity, timestamp).
- **Broker:** Firebase RTDB stores and automatically distributes updates.
- **Subscriber:** The web app uses Firebase's onValue() listener to instantly reflect new data.

This eliminates traditional "polling" latency and demonstrates modern **real-time web architecture**.

4.2 Anonymous Authentication

Both the ESP8266 and the web dashboard use Firebase Anonymous Authentication to satisfy Realtime Database rules:

```
{
  "rules": {
    ".read": "auth != null",
    ".write": "auth != null"
  }
}
```

4.3 The ESP8266 System-on-Chip (SoC) Capability

The NodeMCU board is built around the **ESP8266 System-on-Chip (SoC)**, which fundamentally combines a powerful processing unit with integrated Wi-Fi capabilities.

- **Integrated Functions:** The ESP8266 is not just a microcontroller; it's a complete networking solution. It integrates the 32-bit Tensilica L106 microcontroller core, SRAM, and the essential components for Wi-Fi (802.11 b/g/n) onto a single chip.
- **Microcontroller Role:** It runs the application code (reading the DHT sensor) and manages the hardware interrupts.
- **Networking Role:** It handles the entire TCP/IP stack, allowing it to connect to the local network and establish secure, outbound connections (HTTPS/WebSockets) to the Firebase cloud services, acting as a true edge device for the IoT system.

4.4 DHT One-Wire Communication Protocol

The DHT sensor (DHT11 or DHT22) utilizes a simple but specific **custom single-wire protocol** for data transfer to the host microcontroller (NodeMCU).

- **Data Line:** Only one physical data pin is used for bidirectional communication between the sensor and the MCU.
- **Start Signal:** The process begins when the NodeMCU pulls the data line low for a specific duration (the "start signal"). The DHT sensor then acknowledges this signal and takes control of the line.
- **Data Transmission:** The sensor transmits 40 bits of data (16 bits for humidity, 16 bits for temperature, and 8 bits for a checksum) as a sequence of high and low pulses. The time duration of the high pulse dictates whether the bit is a logic '0' or a logic '1'.
- **Precision and Timing:** This protocol requires precise timing control by the microcontroller, making the use of dedicated libraries (`DHT.h`) essential to handle the intricate timing requirements and calculate the data checksum to ensure accuracy

4.5 Data Structuring and Denormalization in RTDB

Firebase Realtime Database (RTDB) is a NoSQL (JSON-tree structure) database. Effective data storage is critical for performance.

- **Denormalization:** Unlike relational databases, RTDB favors **data denormalization** to speed up read operations. In this project, all related weather readings (`temperature`, `humidity`, `lastUpdated`) are grouped under a single parent node (`/DHT`).
- **Efficient Reading:** By grouping data, the web app can subscribe to the single `/DHT` path using one `onValue()` listener. This minimizes the number of database calls and reduces bandwidth, as only one notification is sent when *any* item under the `/DHT` path changes, making the real-time updates extremely efficient.

4.6 The WebSocket Protocol for Real-Time Streaming

The zero-latency experience of the web dashboard is achieved using the **WebSocket protocol**, which is Firebase Realtime Database's underlying transport mechanism.

- **Traditional HTTP:** Relies on a **Request/Response** model, requiring the client to constantly *poll* the server for new data, leading to high latency and inefficiency.
- **WebSocket:** Establishes a **persistent, full-duplex communication channel** over a single TCP connection. Once the connection is open, the Firebase Broker (Cloud) can instantly *push* new data to the Web App (Subscriber) the moment the NodeMCU (Publisher) writes it, completely eliminating polling overhead.

5. Implementation and Code

5.1 Firmware (NodeMCU C++) — Publisher

The **NodeMCU Firmware** (main.cpp) is responsible for hardware interaction and data publication. It uses the Firebase_ESP_Client library to sign in anonymously and transmits temperature, humidity, and a timestamp to the Realtime Database every five seconds.

Link to code :

https://github.com/veduuuuuuu/esp32-realtime-weather-dashboard/blob/main/esp32_firmware/esp32_firmware.ino

5.2 Web Application (HTML + JavaScript) — Subscriber

The **Web Application** (index.html) is a single-file, fully responsive dashboard built with HTML, Tailwind CSS for styling, and JavaScript for real-time data handling. It utilizes the Firebase SDK's onValue() listeners to ensure the UI updates instantly upon data changes pushed by the NodeMCU.

Link to code :

https://github.com/veduuuuuuu/esp32-realtime-weather-dashboard/blob/main/public/index.html

6. Results and Conclusion

6.1 Results

- The NodeMCU successfully connected to Firebase and transmitted live sensor data every 5 seconds.
- The hosted web app (Firebase Hosting) displayed real-time updates without refreshing.
- Latency observed was less than **500ms**, demonstrating effective Pub/Sub operation.
- Both temperature and humidity were dynamically updated in the browser UI.

Realtime Database Snapshot:

```
DHT
 ├── humidity: 62.3
 ├── temperature: 27.8
 └── lastUpdated: 742356 // Example Firebase timestamp
```

6.2 Conclusion

This project demonstrates a robust, low-cost, **real-time IoT weather monitoring system** using a fully serverless architecture.

By leveraging Firebase's authentication and real-time capabilities, the system eliminates traditional backend overhead, achieving instant updates from device to user interface. The integration of embedded programming, cloud connectivity, and modern web technologies validates the potential of IoT in scalable, cloud-driven applications.